

Image Transformations + Warping

*The
Ambassador
s (Holbein),
1533*



CS180: Intro to Computer Vision and Comp. Photo
Alexei Efros & Anjoo Kanazawa, UC Berkeley, Fall 2025

Logistics

If you are CE who made it into class, but originally wanted to get into CS280A let me know.

Today you will learn how to do this:



Or this:



Or this!



You will learn how to make panoramas

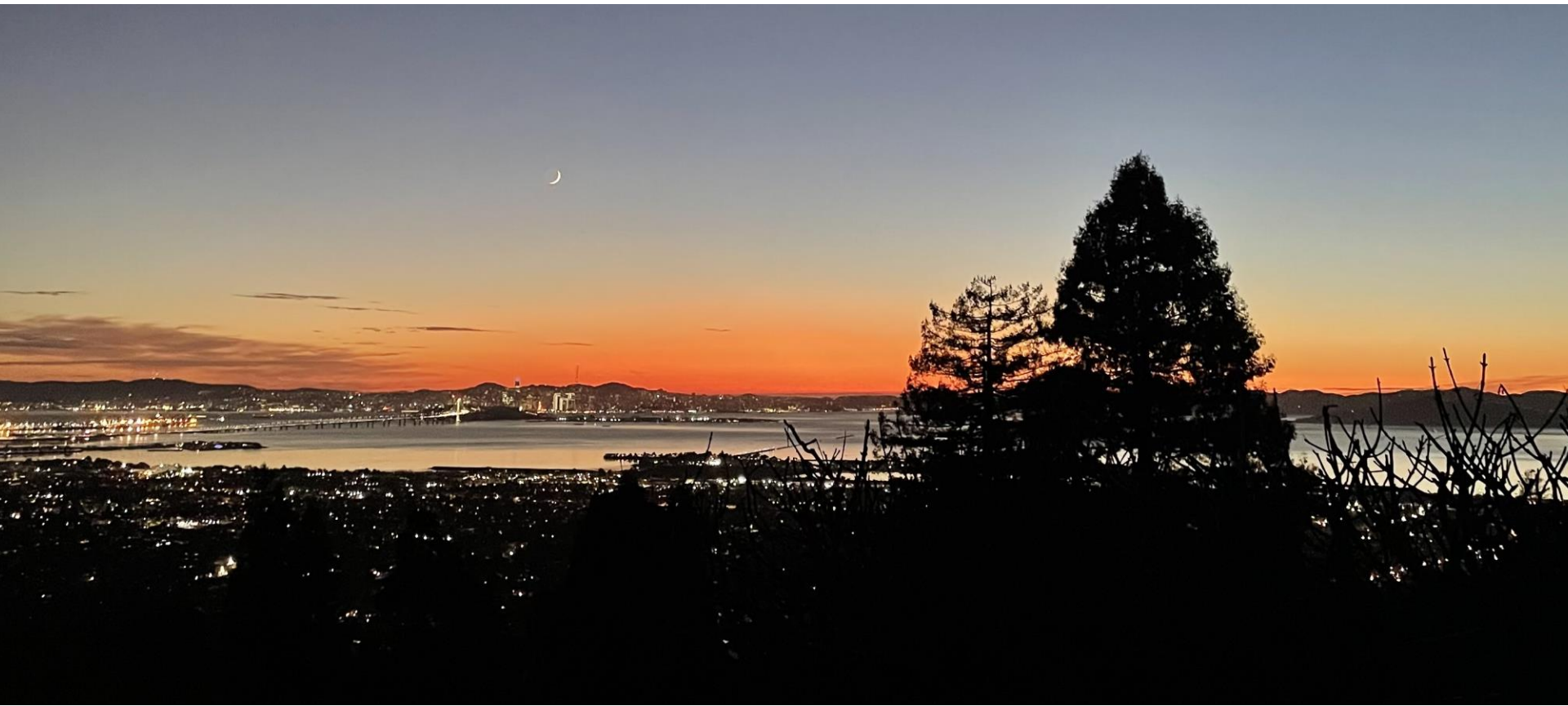


Image Transformations

image filtering: change **range** of image

$$g(x) = T(f(x))$$

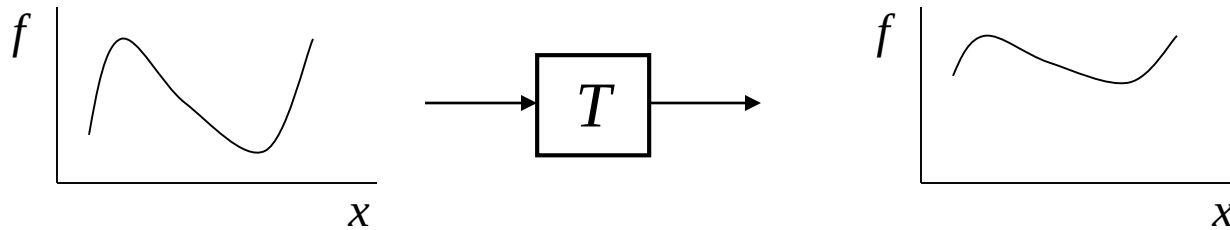


image warping: change **domain** of image

$$g(x) = f(T(x))$$

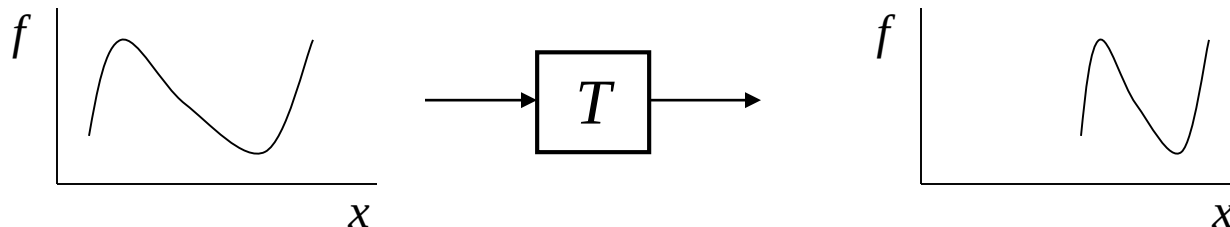


Image Transformations

image filtering: change **range** of image

$$g(x) = T(f(x))$$

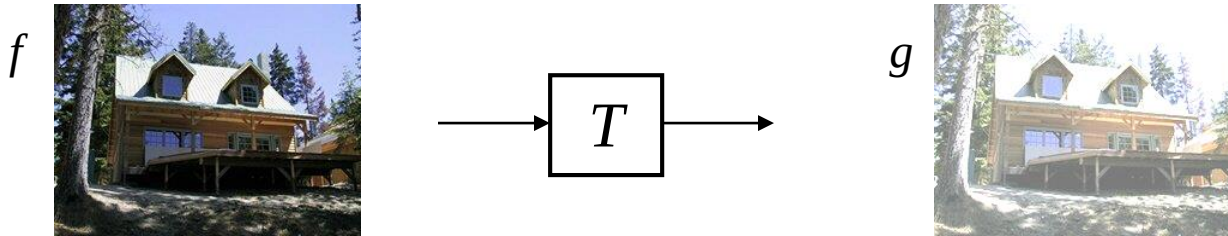
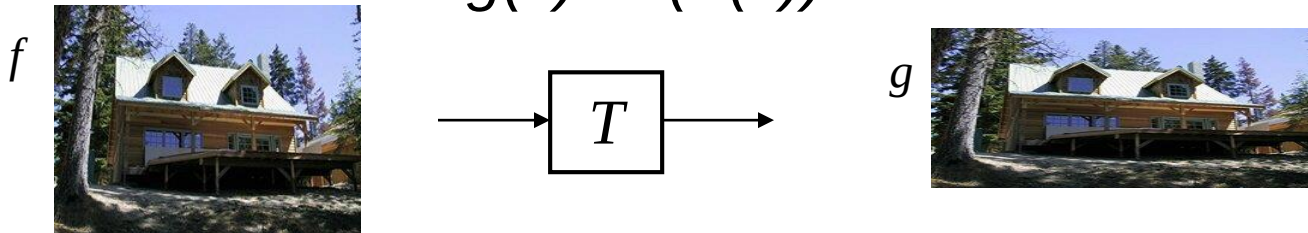


image warping: change **domain** of image

$$g(x) = f(T(x))$$



Parametric (global) warping

Examples of parametric warps:



translation



rotation



aspect



affine



perspective

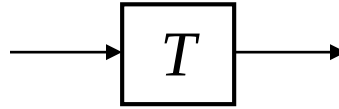


cylindrical

Parametric (global) warping



$$\mathbf{p} = (x, y)$$



$$\mathbf{p}' = (x', y')$$

Transformation T is a coordinate-changing machine:

$$\mathbf{p}' = T(\mathbf{p})$$

What does it mean that T is global?

- Is the same for any point p
- can be described by just a few numbers (parameters)

Let's represent a linear T as a matrix:

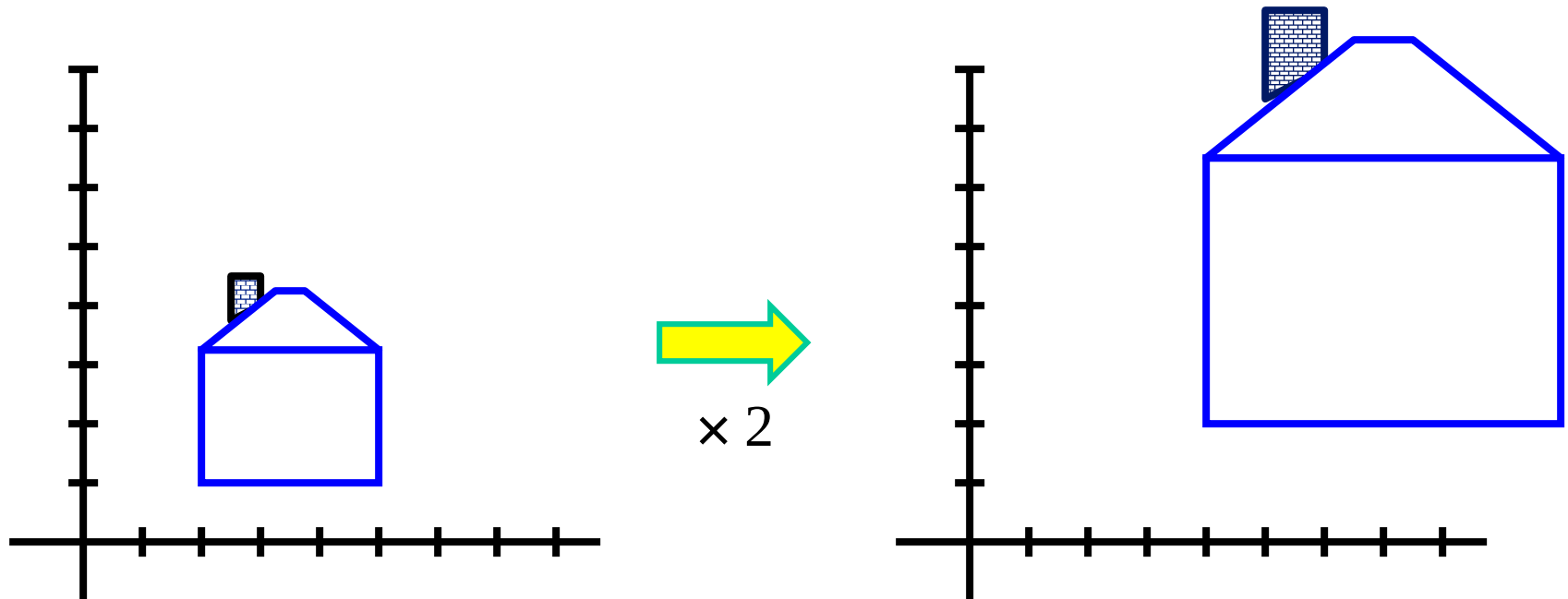
$$\mathbf{p}' = \mathbf{M}\mathbf{p}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \mathbf{M} \begin{bmatrix} x \\ y \end{bmatrix}$$

Scaling

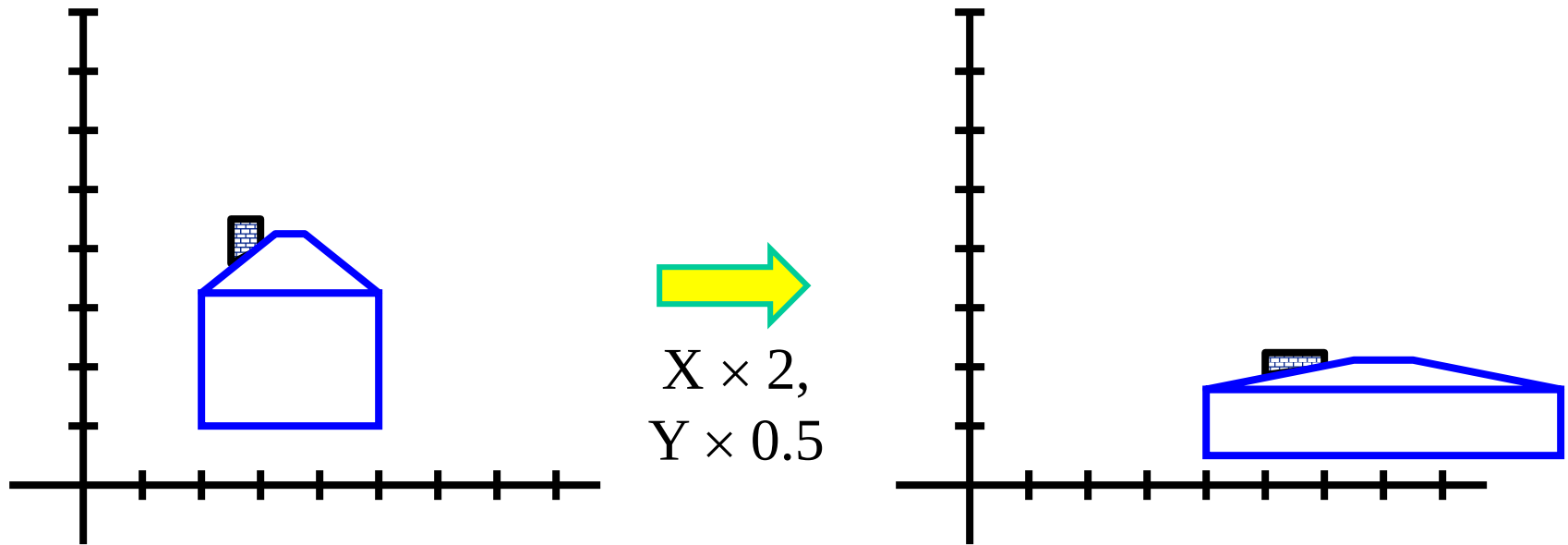
Scaling a coordinate means multiplying each of its components by a scalar

Uniform scaling means this scalar is the same for all components:



Scaling

Non-uniform scaling: different scalars per component:



Scaling

Scaling operation:

$$x' = ax$$

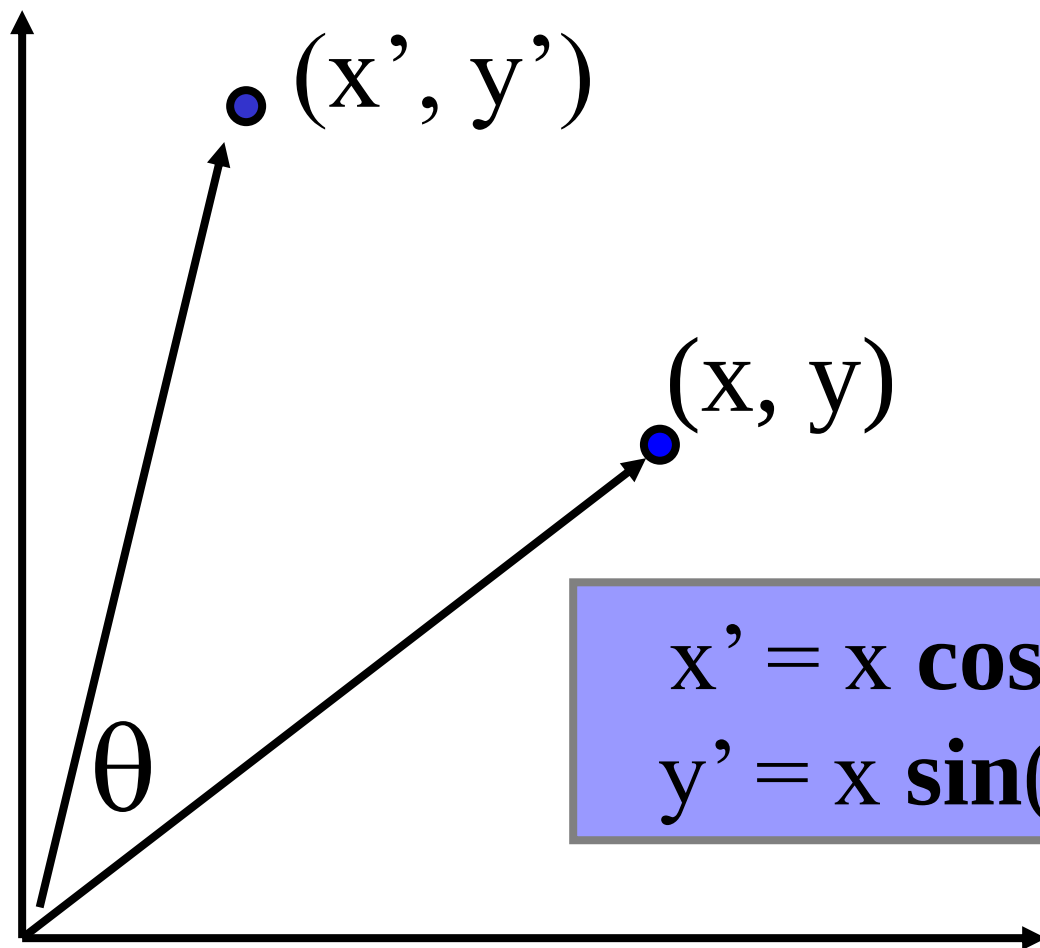
$$y' = by$$

Or, in matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}}_{\text{scaling matrix } S} \begin{bmatrix} x \\ y \end{bmatrix}$$

What's inverse of S?

2-D Rotation



$$\begin{aligned}x' &= x \cos(\theta) - y \sin(\theta) \\ y' &= x \sin(\theta) + y \cos(\theta)\end{aligned}$$

2-D Rotation

This is easy to capture in matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}}_{\mathbf{R}} \begin{bmatrix} x \\ y \end{bmatrix}$$

Is this a linear transformation?

Even though $\sin(\theta)$ and $\cos(\theta)$ are nonlinear functions of θ ,

- ***x' is a linear combination of x and y***
- ***y' is a linear combination of x and y***

What is the inverse transformation?

- Rotation by $-\theta$ $\mathbf{R}^{-1} = \mathbf{R}^T$
- For rotation matrices

2x2 Matrices

What types of transformations can be represented with a 2x2 matrix?

2D Identity?

$$\begin{aligned}x' &= x \\ y' &= y\end{aligned}\quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

2D Scale around (0,0)?

$$\begin{aligned}\mathbf{x}' &= \mathbf{s}_x * \mathbf{x} \\ \mathbf{y}' &= \mathbf{s}_y * \mathbf{y}\end{aligned}\quad \begin{bmatrix} \mathbf{x}' \\ \mathbf{y}' \end{bmatrix} = \begin{bmatrix} \mathbf{s}_x & 0 \\ 0 & \mathbf{s}_y \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \end{bmatrix}$$

2x2 Matrices

What types of transformations can be represented with a 2x2 matrix?

2D Rotate around (0,0)?

$$\begin{aligned}x' &= \cos \Theta * x - \sin \Theta * y \\ y' &= \sin \Theta * x + \cos \Theta * y\end{aligned}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \Theta & -\sin \Theta \\ \sin \Theta & \cos \Theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

2D Shear?

$$\begin{aligned}x' &= x + sh_x * y \\ y' &= sh_y * x + y\end{aligned}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & sh_x \\ sh_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

2x2 Matrices

What types of transformations can be represented with a 2x2 matrix?

2D Mirror about Y axis?

$$\begin{aligned}x' &= -x \\ y' &= y\end{aligned}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



Reflection through (0,0)?

$$\begin{aligned}x' &= -x \\ y' &= -y\end{aligned}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



All 2D Linear Transformations

Linear transformations are combinations of ...

- Scale,
- Rotation,
- Shear, and
- Mirror/Reflection

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Properties of linear transformations:

- Origin maps to origin
- Lines map to lines
- Parallel lines remain parallel
- Ratios are preserved
- Closed under composition

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} e & f \\ g & h \end{bmatrix} \begin{bmatrix} i & j \\ k & l \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

2x2 Matrices

What types of transformations can be represented with a 2x2 matrix?

2D Translation?

$$\mathbf{x}' = \mathbf{x} + \mathbf{t}_x \quad \text{NO!}$$

$$\mathbf{y}' = \mathbf{y} + \mathbf{t}_y$$

Only linear 2D transformations
can be represented with a 2x2 matrix

Homogeneous Coordinates

Q: How can we represent translation as a 3x3 matrix?

$$x' = x + t_x$$

$$y' = y + t_y$$

A: Using the rightmost column:

$$\mathbf{Translation} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Homogeneous Coordinates

Homogeneous coordinates

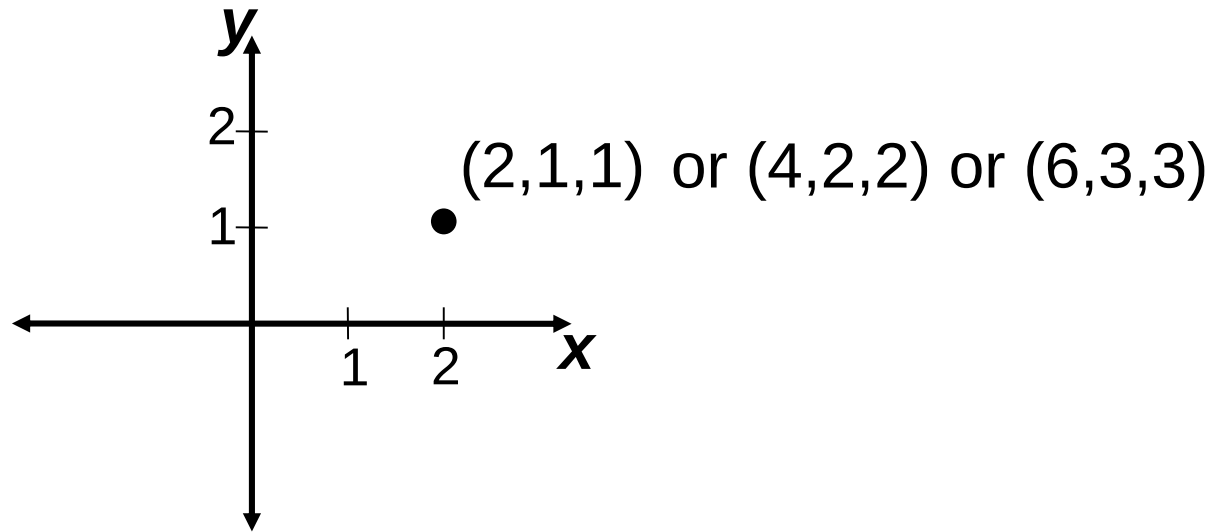
- represent coordinates in 2 dimensions with a 3-vector

$$\begin{bmatrix} x \\ y \end{bmatrix} \xrightarrow{\text{homogeneous coords}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Homogeneous Coordinates

Add a 3rd coordinate to every 2D point

- (x, y, w) represents a point at location $(x/w, y/w)$
- $(x, y, 0)$ represents a point at infinity
- $(0, 0, 0)$ is not allowed



Convenient tool to
express translation
as a linear transform

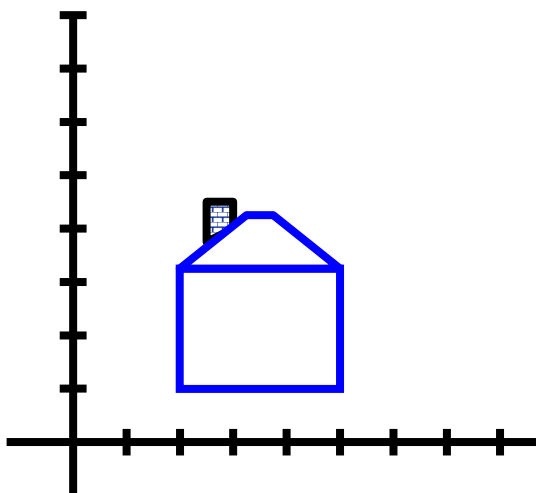
Translation

Example of translation

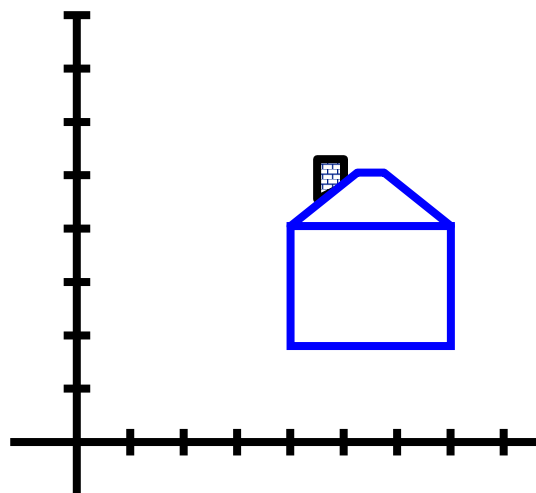
Homogeneous Coordinates



$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$



$$\begin{aligned} t_x &= 2 \\ t_y &= 1 \end{aligned}$$



Basic 2D Transformations

Basic 2D transformations as 3x3 matrices

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Translate

$$\begin{bmatrix} \mathbf{x}' \\ \mathbf{y}' \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{s}_x & 0 & 0 \\ 0 & \mathbf{s}_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ 1 \end{bmatrix}$$

Scale

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Rotate

$$\begin{bmatrix} \mathbf{x}' \\ \mathbf{y}' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & \mathbf{sh}_x & 0 \\ \mathbf{sh}_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ 1 \end{bmatrix}$$

Shear

Matrix Composition

Transformations can be combined by
matrix multiplication

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left(\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

p' = **T**(t_x,t_y) **R**(Θ) **S**(s_x,s_y) **p**

Does the order of multiplication matter?

Affine Transformations

Affine transformations are combinations of ...

- Linear transformations, and
- Translations

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Properties of affine transformations:

- Origin does not necessarily map to origin (translation!)
- Lines map to lines
- Parallel lines remain parallel
- Ratios are preserved
- Closed under composition
- Models change of basis

Will the last coordinate w always be 1?

Projective Transformations/Homography

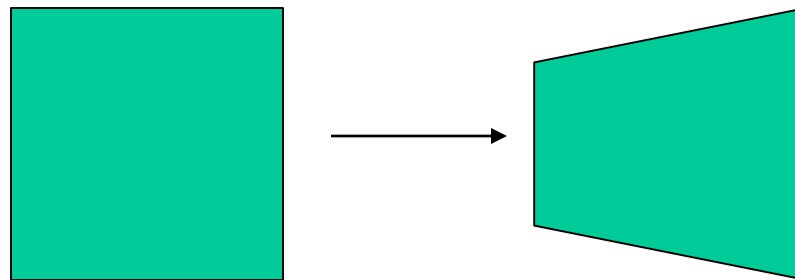
Projective transformations ...

- Affine transformations, and
- Projective warps

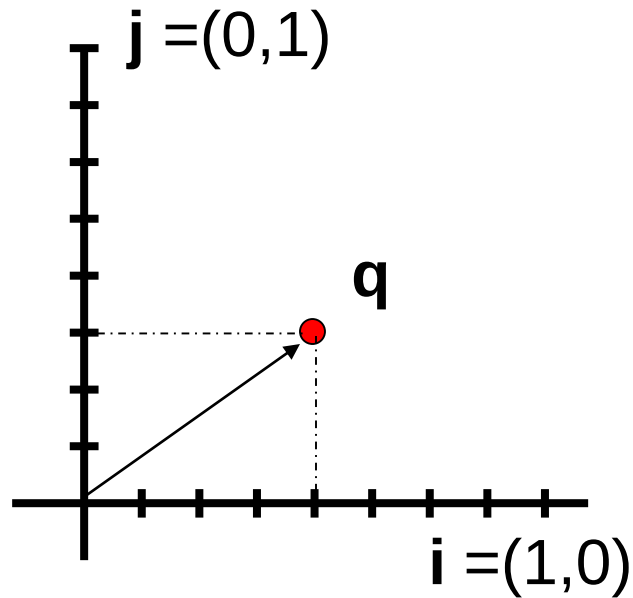
$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Properties of projective transformations:

- Origin does not necessarily map to origin
- Lines map to lines
- Parallel lines do not necessarily remain parallel
- Ratios are not preserved
- Closed under composition
- Models change of basis

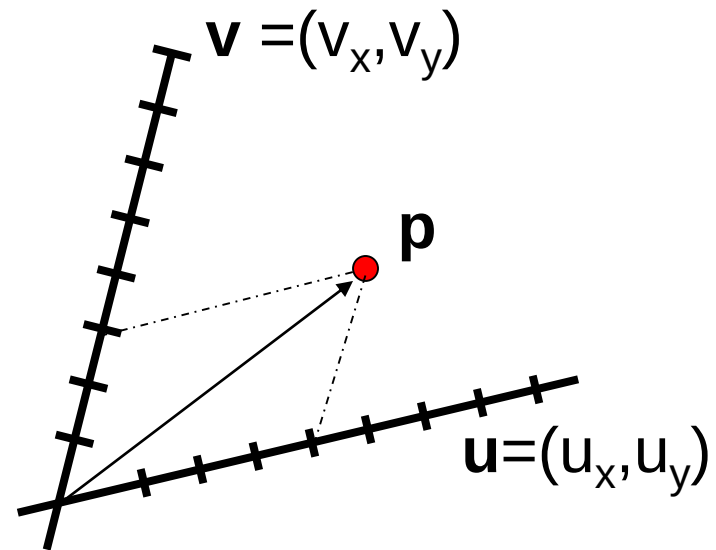


Another way to think about transformation



$$\mathbf{q} = 4\mathbf{i} + 3\mathbf{j} = (4, 3)$$

ij-basis

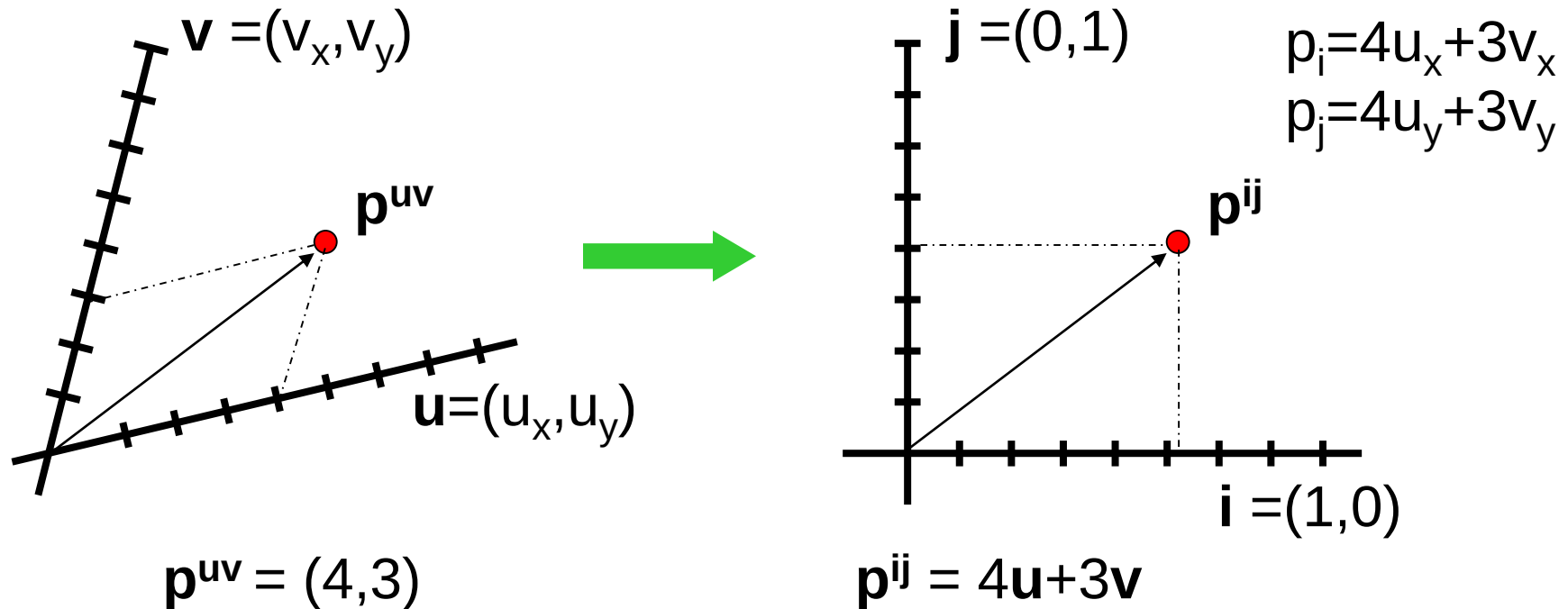


$$\mathbf{p} = 4\mathbf{u} + 3\mathbf{v}$$

uv-basis

Same point represented using a Different Basis

Linear Transformations as Change of Basis

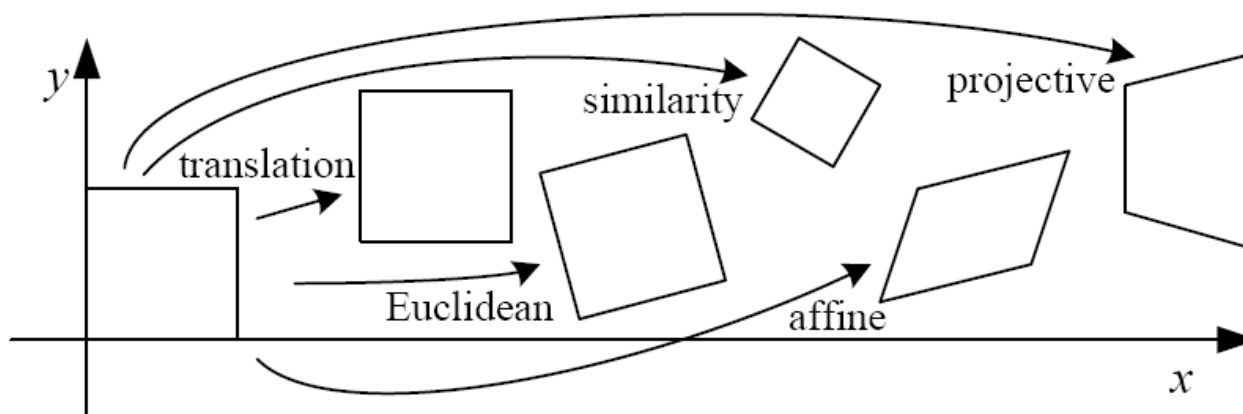


$$\mathbf{p}^{ij} = \begin{bmatrix} u_x & v_x \\ u_y & v_y \end{bmatrix} \begin{bmatrix} 4 \\ 3 \end{bmatrix} = \begin{bmatrix} u_x & v_x \\ u_y & v_y \end{bmatrix} \mathbf{p}^{uv}$$

Transformation from uv to ij basis

Any linear transformation is a basis!!!

2D image transformations



Name	Matrix	# D.O.F.	Preserves:	Icon
translation	$\begin{bmatrix} \mathbf{I} & \mathbf{t} \end{bmatrix}_{2 \times 3}$			
rigid (Euclidean)	$\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$			
similarity	$\begin{bmatrix} s\mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$			
affine	$\begin{bmatrix} \mathbf{A} \end{bmatrix}_{2 \times 3}$			
projective	$\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}_{3 \times 3}$			

These transformations are a nested set of groups

- Closed under composition and inverse is a member



Image Transforms in Biology

D'Arcy Thompson

<http://www-groups.dcs.st-and.ac.uk/~history/Miscellaneous/darcy.html>

http://en.wikipedia.org/wiki/D'Arcy_Thompson

Importance of shape and structure in evolution

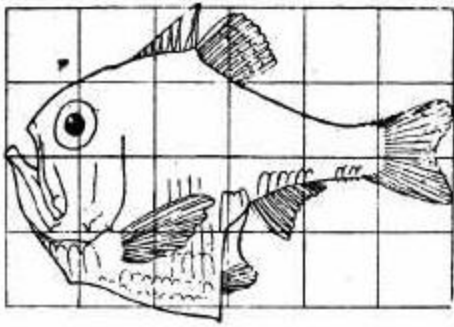
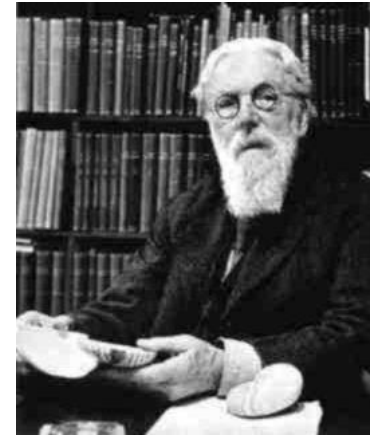
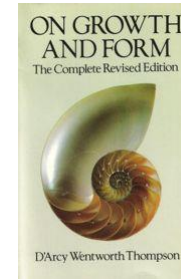


Fig. 517. *Argyropelecus Olfersi*.

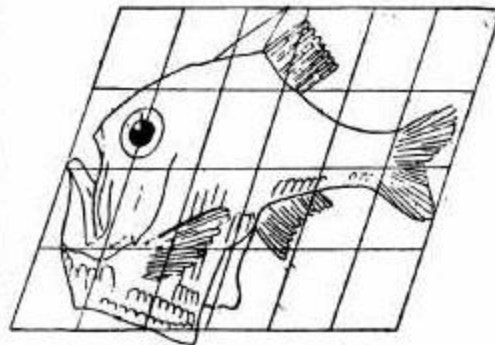
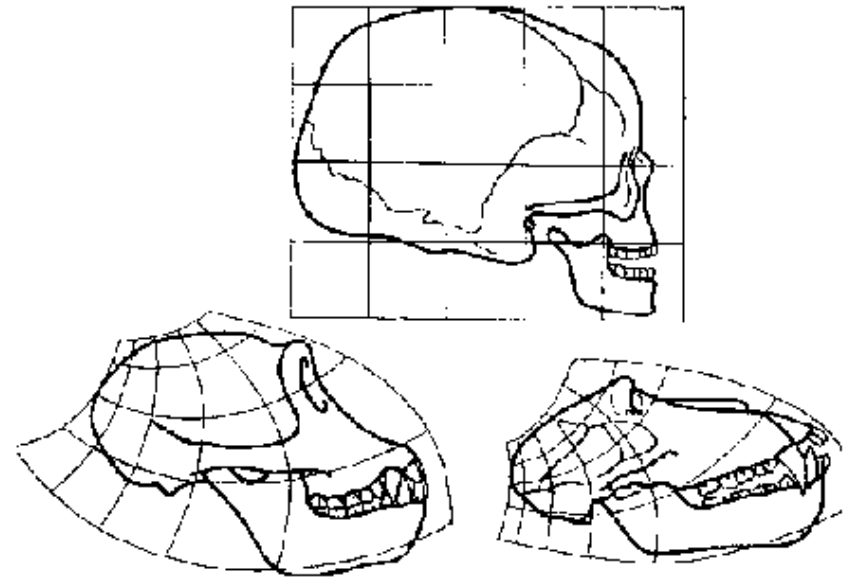


Fig. 518. *Sternoptyx diaphana*.



Skulls of a human, a chimpanzee and a baboon and transformations between them

Now the fun stuff

How do we go from this to this image??



Picture of Ayuna when she was a baby

Break

Now the fun stuff

How do we go from this to this image??



Now the fun stuff

How do we go from this to this image??



Picture of Ayuna when she was a baby

Now the fun stuff

How do we go from this to this image??

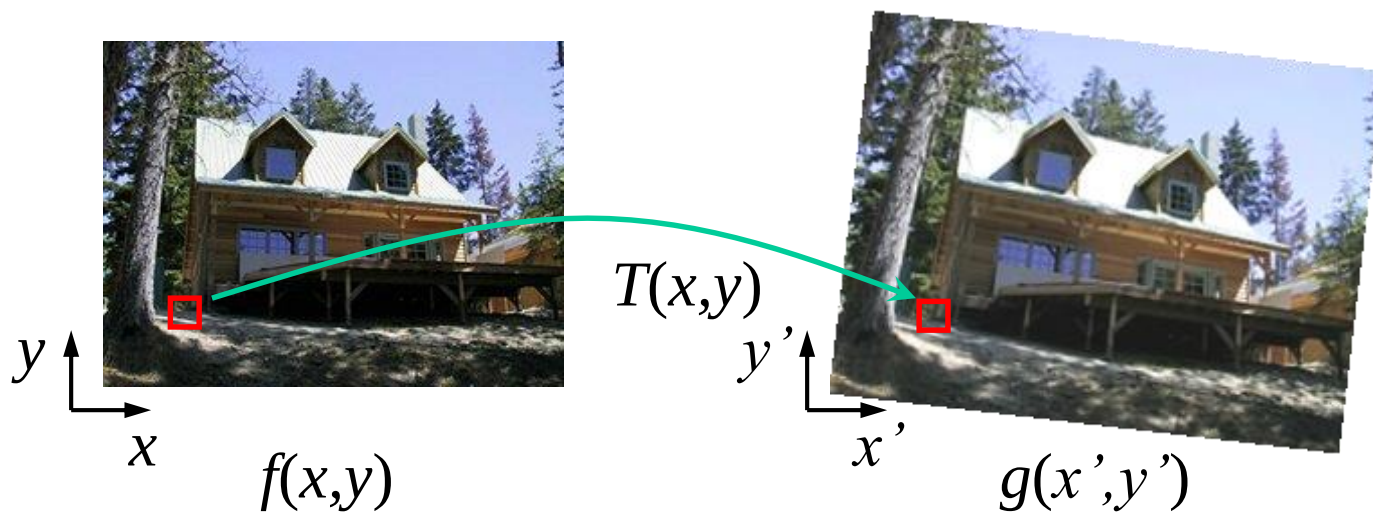


Crop +
enlarge



Picture of Ayuna when she was a baby

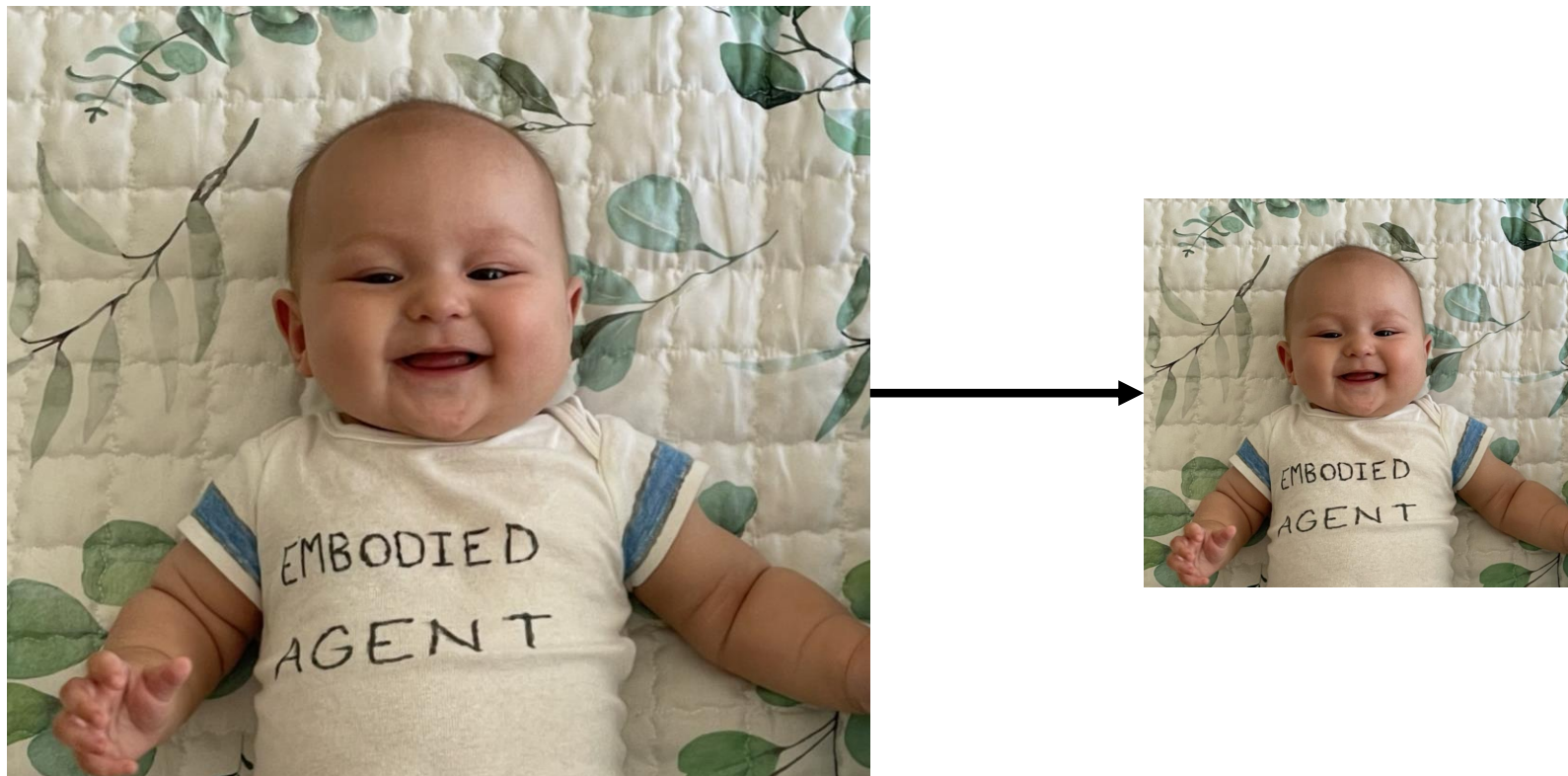
Warping Pixels



Given a coordinate transform $(x',y') = T(x,y)$ and a source image $f(x,y)$, how do we compute a transformed image $g(x',y') = f(T(x,y))$?

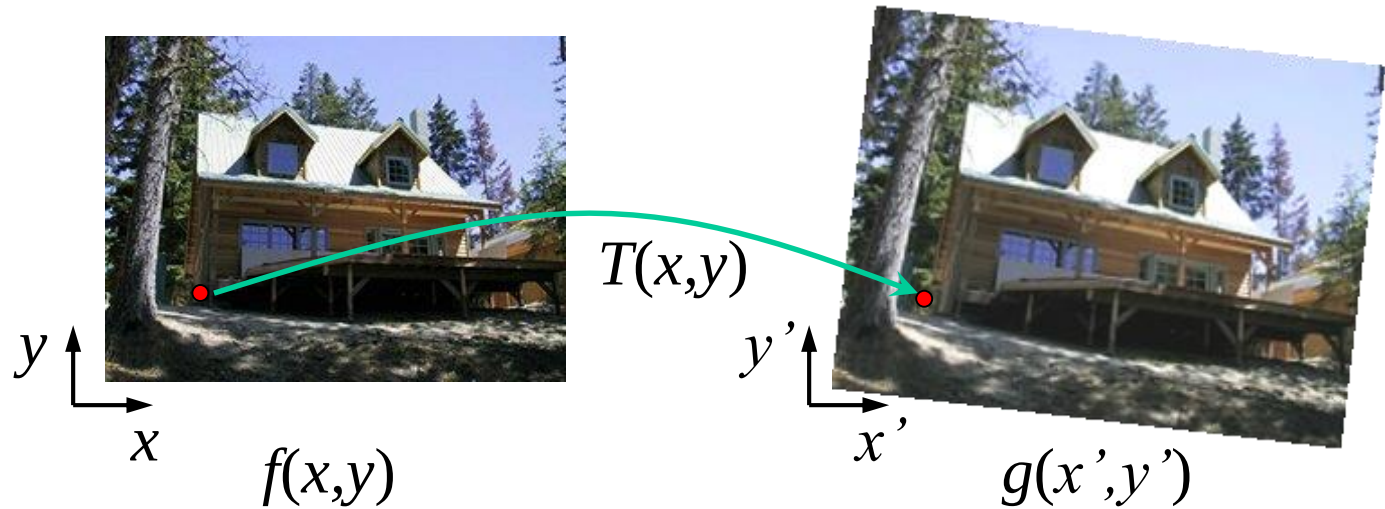
The Magic of imresize

So how to do it? Say the goal is to reduce the image size exactly by half



- Take every other pixel!
- This is like transforming every other pixel to a new location

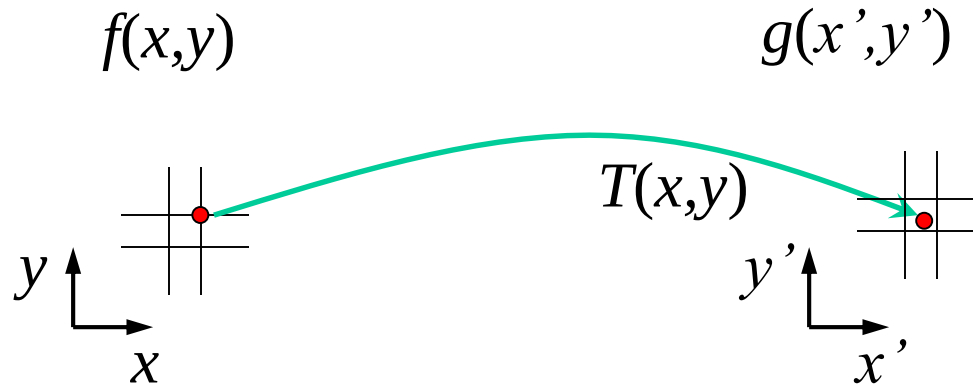
Forward warping



Send each pixel $f(x,y)$ to its corresponding location
 $(x',y') = T(x,y)$ in the second image

Q: what if pixel lands “between” two pixels?

Forward warping



Send each pixel $f(x,y)$ to its corresponding location
 $(x',y') = T(x,y)$ in the second image

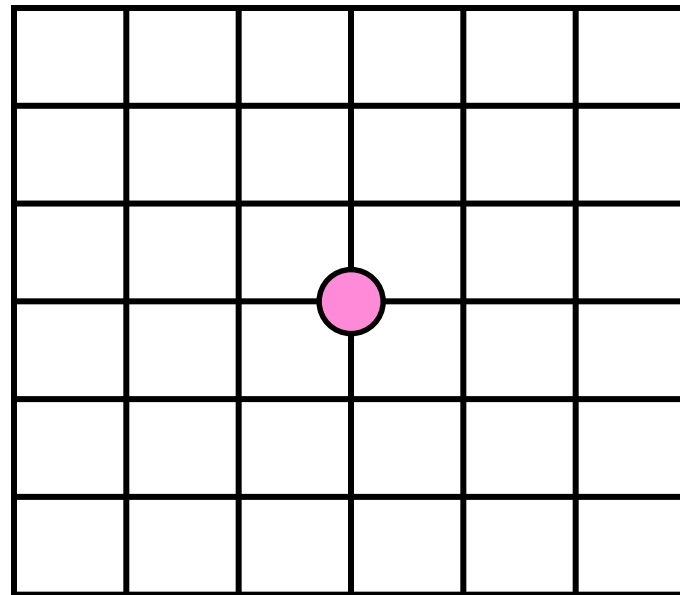
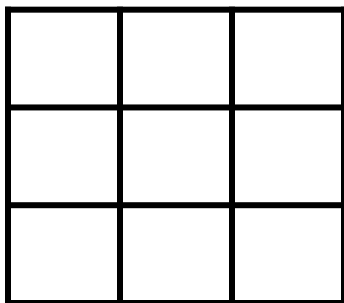
Q: what if pixel lands “between” two pixels?

A: distribute color among neighboring pixels (x',y')

- Known as “splatting” (Check out `griddata` in Matlab)
- Generally, a very bad idea. Why?

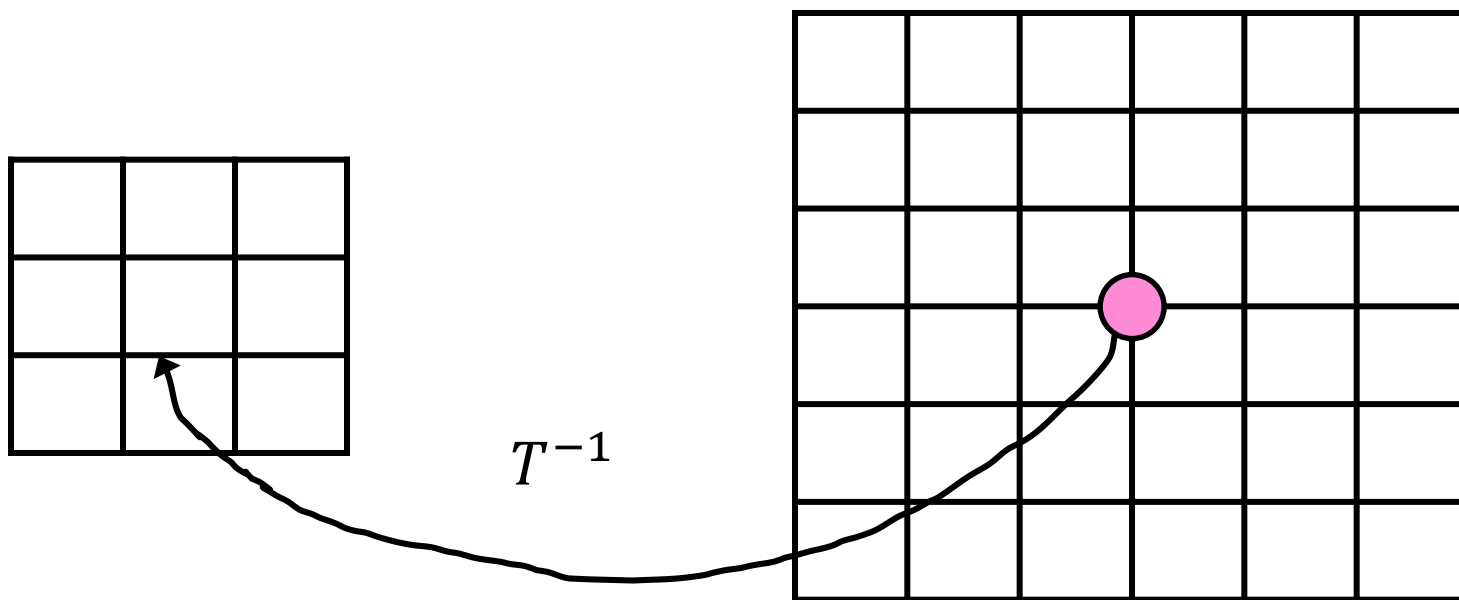
Can we do better?

What if we want to make the image bigger?



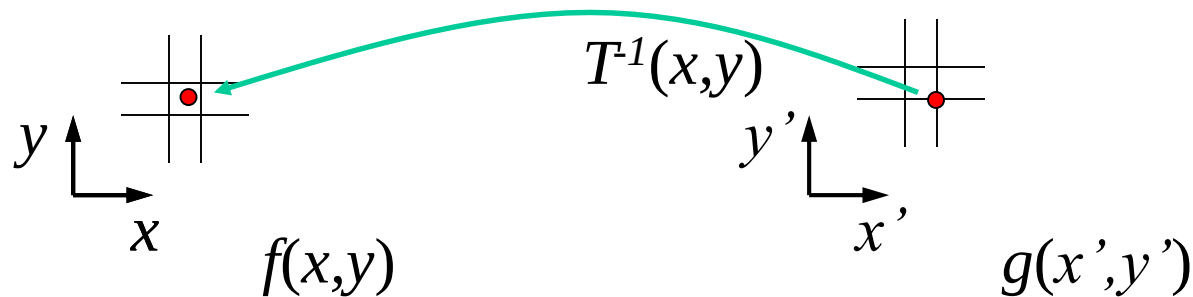
Can we do better?

What if we want to make the image bigger?



Go backwards!

Inverse warping



Get each pixel $g(x', y')$ from its corresponding location $(x, y) = T^{-1}(x', y')$ in the first image

Q: what if pixel comes from “between” two pixels?

A: *Interpolate* color value from neighbors

- nearest neighbor, bilinear, Gaussian, bicubic
- Check out `interp2` in Matlab / Python

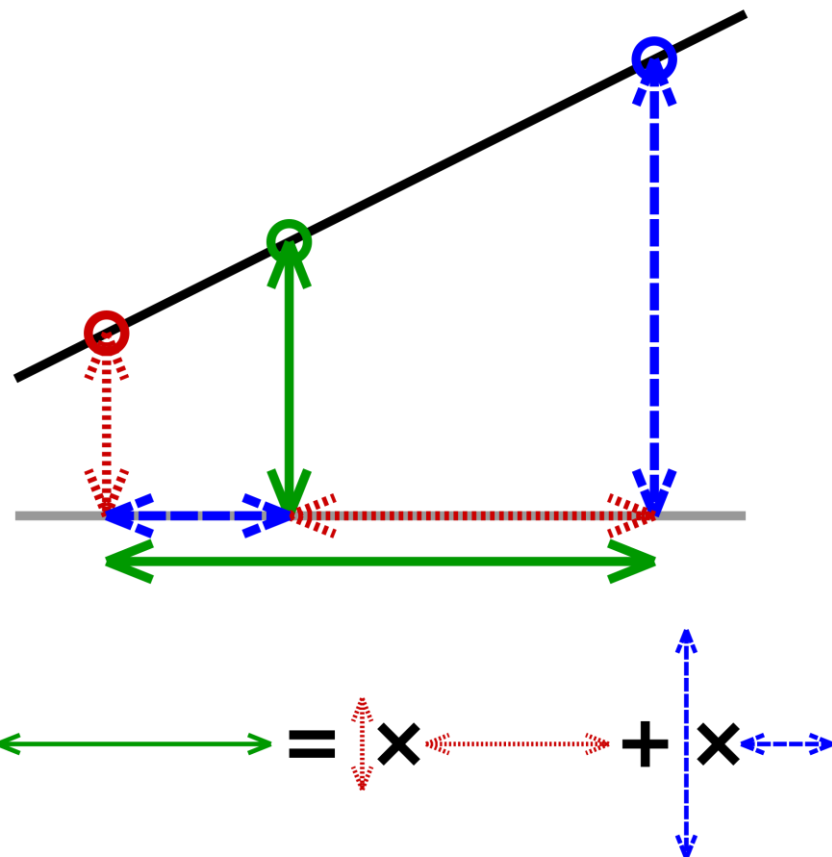
What do we need to watch out for??

Step 0 of any sampling:

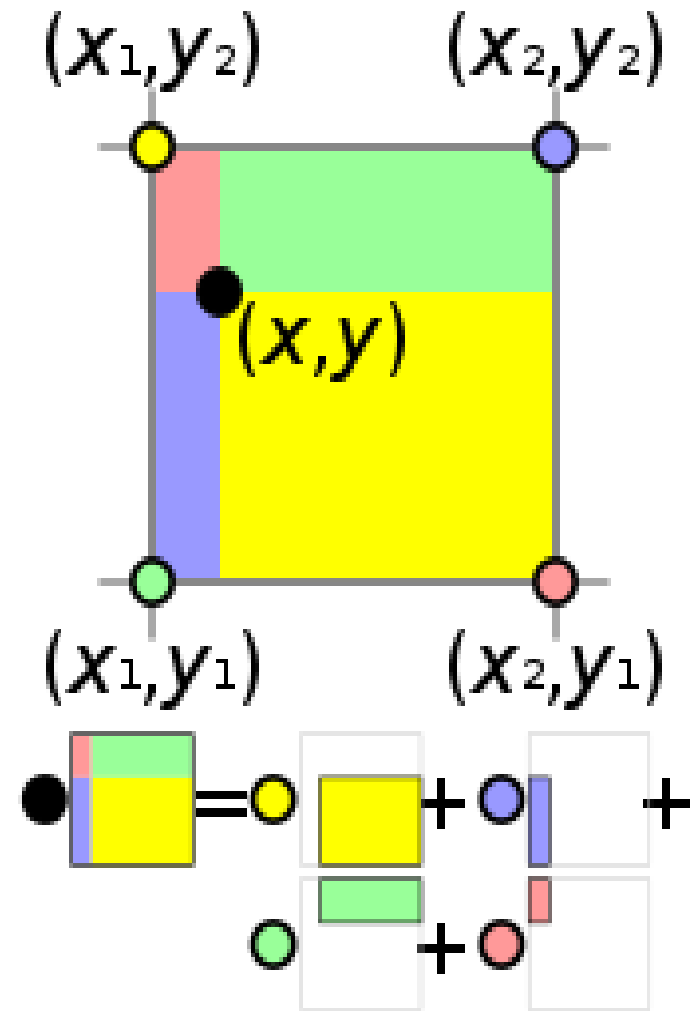
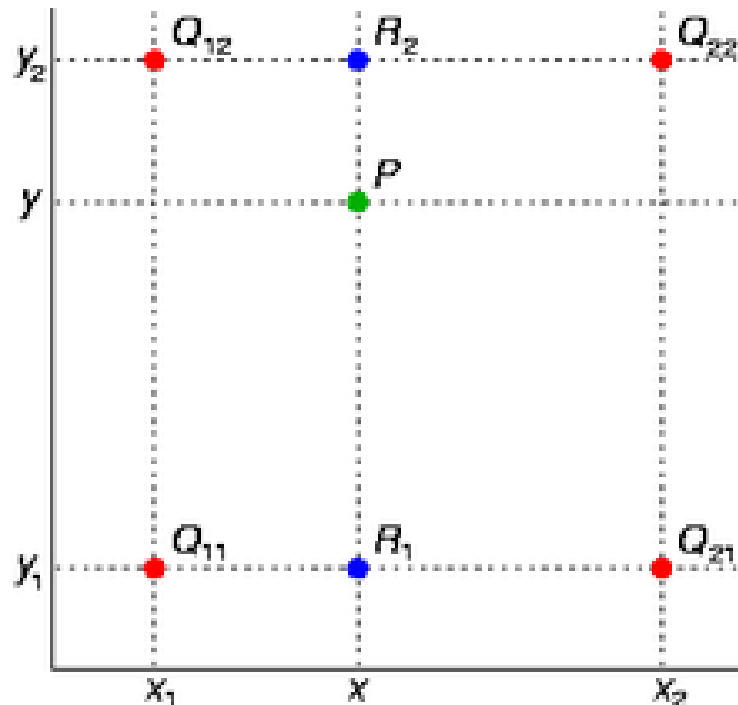
Aliasing!!

1. Gaussian filter
2. Find the inverse warping
3. Interpolate

Recall 1D interpolation:

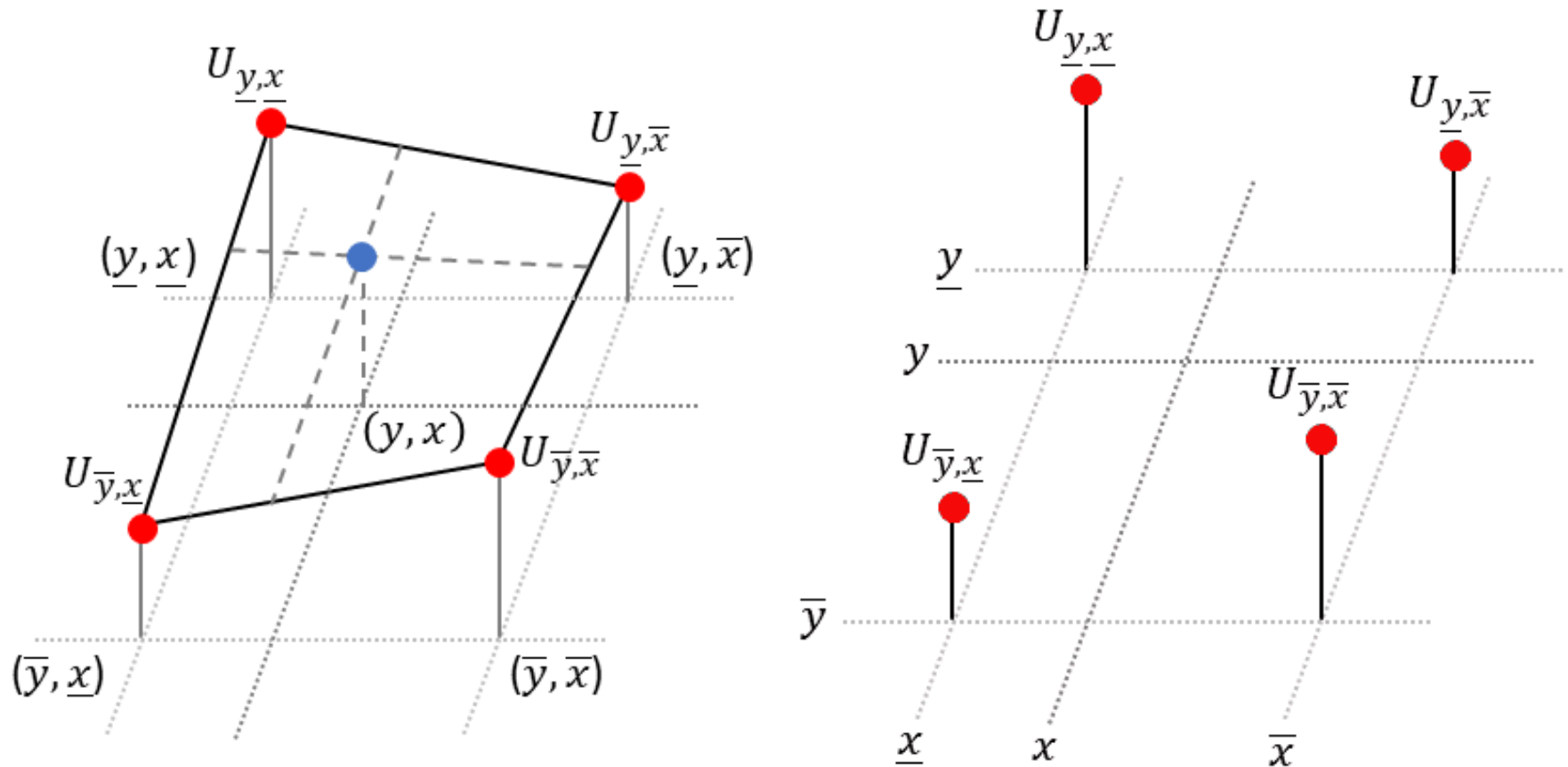


Bilinear Interpolation

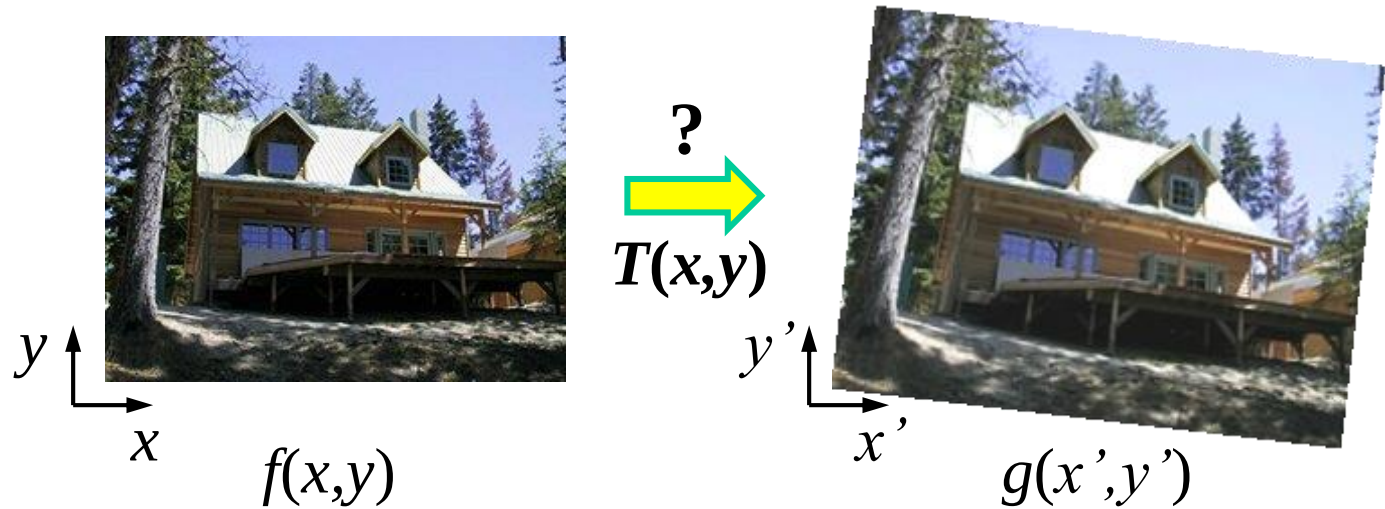


http://en.wikipedia.org/wiki/Bilinear_interpolation
 Help interp2

Bilinear Interpolation



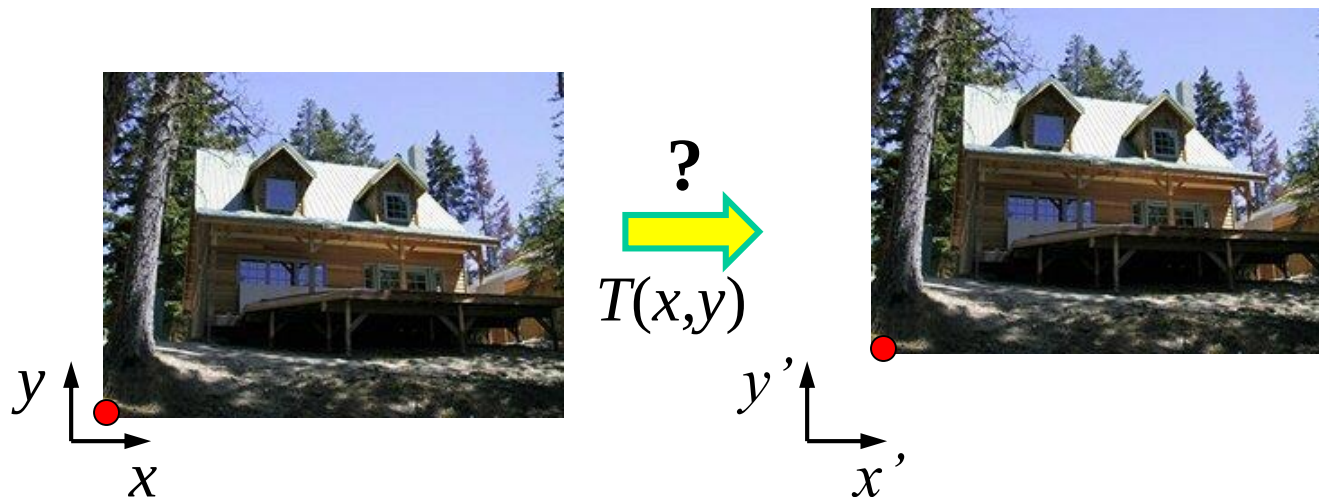
Recovering Transformations



What if we know f and g and want to recover the transform T ?

- e.g. better align images from Project 1
- willing to let user provide correspondences
 - How many do we need?

Translation: # correspondences?



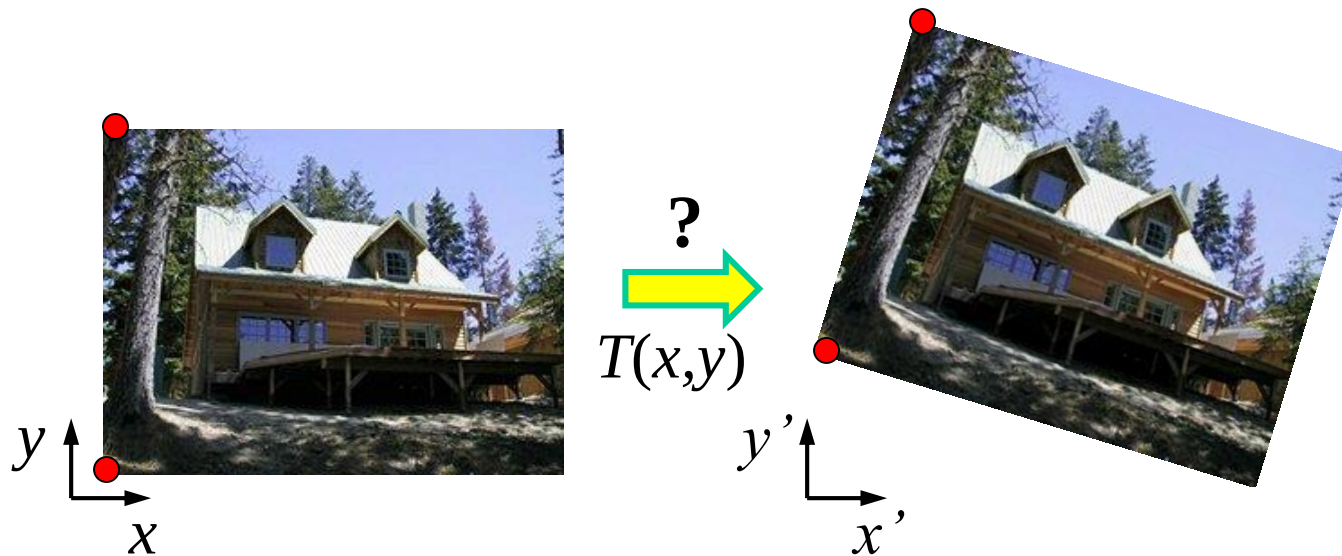
How many correspondences needed for translation?

How many Degrees of Freedom?

What is the transformation matrix?

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & p'_x - p_x \\ 0 & 1 & p'_y - p_y \\ 0 & 0 & 1 \end{bmatrix}$$

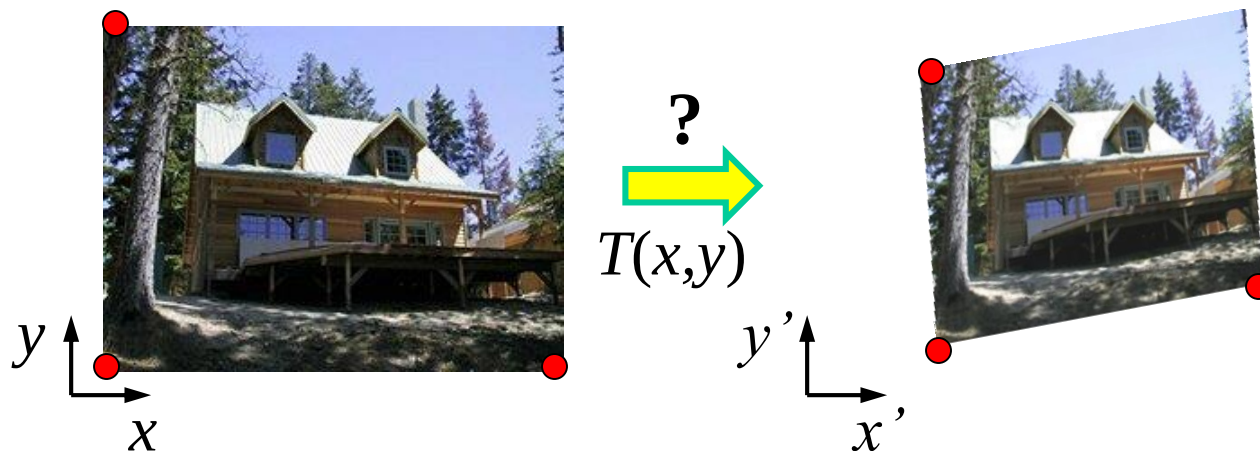
Euclidian: # correspondences?



How many correspondences needed for translation+rotation?

How many DOF?

Affine: # correspondences?

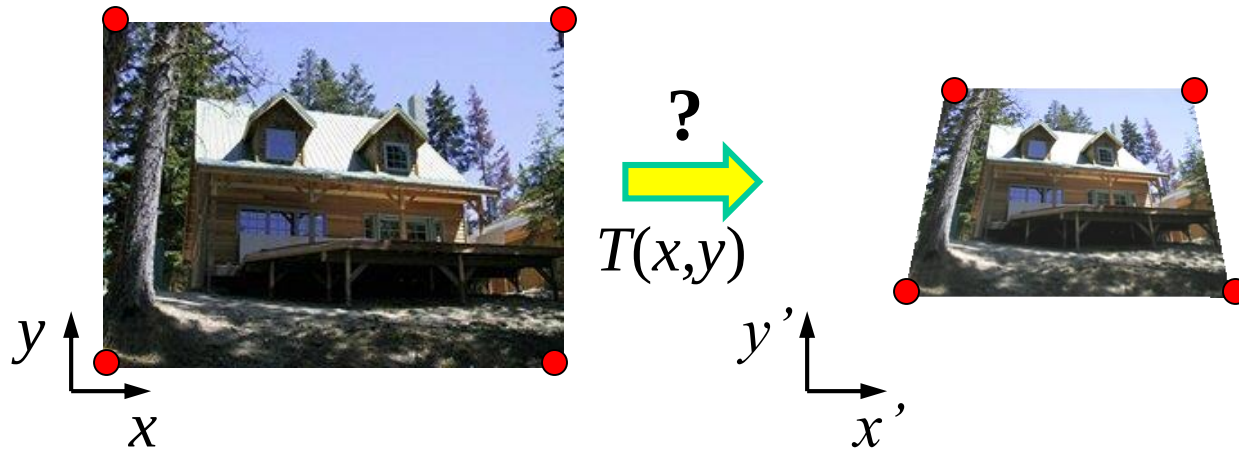


How many correspondences needed for affine?

How many DOF?

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Projective: # correspondences?



How many correspondences needed for projective/homography?

How many DOF?

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Next project: Panorama stitching

1. Compute the homography
2. Warp the image

